Evaluating and Improving Chunking Methods for Search and Retrieval-Augmented Generation (RAG) Systems

Shelly Schwartz
University of Virginia
School of Data Science
Charlottesville, Virginia
tfk6ua@virginia.edu

***Abstract*—Chunking partitions documents into segments for retrieval and Retrieval-Augmented Generation (RAG). Fixed-size and sentence-level heuristics ignore semantic boundaries and produce incoherent or redundant segments that reduce retrieval precision. We propose a recursive chunker that places boundaries at the strongest local discontinuities of contextual token embeddings under a maximum-length constraint, and we describe a dynamic programming alternative that we deprioritize on compute grounds and report as a negative result. We further introduce a three-axis intrinsic evaluation protocol (Coherence, Completeness, Relevance Purity) implemented as an LLM-as-a-Judge with deterministic prompts. On four BEIR datasets and four open-weight embedding models, the recursive chunker obtains the strongest or near-strongest extrinsic retrieval scores on SciFact, NFCorpus, and FiQA2018 and the highest intrinsic macro score on the same three datasets; fixed-size chunking is strongest on TREC-COVID. Intrinsic and extrinsic scores correlate at Pearson $r = 0.85$ over 12 paired cells, indicating that the intrinsic protocol supplies a screening signal for chunking quality.**

***Index Terms*—Chunking, Search, Retrieval-Augmented Generation, RAG, Intrinsic Evaluation, Dynamic Programming, LLM-as-a-Judge**

I. Introduction

Chunking determines how documents are represented and retrieved in search and RAG pipelines. Poorly placed boundaries fragment evidence, lower embedding similarity to queries, and propagate errors into generation. Effective chunking preserves logical continuity, avoids redundancy, and produces self-contained units. Most deployed chunkers prioritize simplicity or throughput over either property: fixed-size windows ignore semantic boundaries, and sentence-level segmentation produces fragments of inconsistent informativeness. Recent semantic methods, including Late Chunking [1], CrossFormer [2], S2 Chunking [3], HiChunk [4], and HOPE [5], address parts of the problem but trade off coverage, redundancy, supervision, and compute in different ways. Section II surveys these methods.

This paper makes three contributions. (i) A recursive chunker that places boundaries at the strongest local discontinuities of contextual token embeddings under a maximum-length constraint. (ii) A dynamic programming formulation reported as a negative result on the basis of compute cost and limited gains in early experiments. (iii) A three-axis intrinsic LLM-as-a-Judge protocol consisting of Coherence, Completeness, and Relevance Purity scores. We evaluate on four BEIR datasets and four open-weight embedding models. Recursive chunking is the strongest method on three of four datasets in both extrinsic retrieval and intrinsic scoring; fixed-size chunking is strongest on TREC-COVID. Intrinsic and extrinsic scores correlate at Pearson $r = 0.85$.

A. Confounds in Standard Chunking Evaluation

Chunking quality is typically measured indirectly through retrieval metrics such as nDCG@10, Recall@K, and MRR [6]. Gains on these metrics are commonly attributed to better segmentation, but retrieval performance also depends on the retriever, the embedding model, the ranking function, and the query distribution, none of which are properties of the chunker. Three additional proxies appear in the literature with similar issues: retrieval precision treated as a direct surrogate; QA answer-span overlap [7], which rewards localized evidence rather than coherent segmentation; and boundary alignment with sentence or paragraph breaks, which captures surface alignment rather than semantic independence. None directly measures whether chunks are self-contained or non-redundant. The intrinsic protocol in Section V-C targets these properties explicitly.

II. Literature Review

A. Late Chunking (Günther et al., 2024)

Late Chunking [1] encodes the full document once and produces chunk embeddings by mean-pooling token representations within each chunk span. Each embedding therefore carries document-level context, improving long-range dependency capture, and Late Chunking reports consistent gains on SciFact, FiQA, and TREC-COVID. The method increases compute relative to chunk-level encoding and does not penalize redundant chunks. It modifies *how* a chunk is represented but not *where* boundaries are placed, so it is orthogonal to the boundary-selection method in Section III-B.

B. Adaptive Chunking (Junior et al., 2026)

Adaptive Chunking [8] selects per-document strategies using intrinsic metrics (ICC, DCC, BI, SC, RC) for intra-chunk coherence, inter-chunk discontinuity, and boundary informativeness. Two of its claims motivate this work: that chunking should be evaluated intrinsically rather than only through downstream retrieval, and that no single strategy is optimal across documents.

C. CrossFormer (Ni et al., 2025)

CrossFormer [2] treats chunking as supervised segmentation. A Cross-Segment Fusion Module learns token-level attention transitions to predict boundaries. The model yields smoother inter-chunk transitions and stronger retrieval accuracy, but requires large labeled datasets and tends to overfit specific text distributions.

D. S2 Chunking (Verma et al., 2025)

S2 Chunking [3] combines structural and semantic signals using layout cues (headings, lists, tables) and embedding similarity. It performs well on semi-structured documents but degrades on unstructured text where formatting metadata is sparse.

E. HiChunk (Lu et al., 2025)

HiChunk [4] models hierarchical relationships across sections, paragraphs, and sentences, enabling multi-scale segmentation. Its HiCBench dataset provides annotated boundaries at multiple granularities, at the cost of intensive annotation and compute.

F. HOPE (Brådland et al., 2025)

HOPE [5] is one of the first intrinsic evaluation frameworks for chunking, quantifying cohesion, task utility, and coherence from embedding statistics. Its scores depend on the underlying embedding model, so absolute values are not comparable across architectures.

G. Synthesis

Existing methods fall along a structural-versus-semantic axis. Structure-aware methods (S2 Chunking, HiChunk) preserve formatting but neglect contextual embedding relationships; embedding-based methods (Late Chunking, CrossFormer) optimize semantic continuity at the cost of redundancy and compute. The recursive chunker proposed below operates on contextual token embeddings and incorporates structural cues, and is evaluated against representatives of both families.

III. Method

We describe two methods. The first is a dynamic programming objective that balances intra-chunk cohesion against inter-chunk separation, reported as a negative result. The second is a recursive splitting procedure that combines contextual token embeddings with structural cues, and is the method evaluated end-to-end in Section VI.

A. Dynamic Programming for Cohesion and Separation

Let E_i denote the embedding of the i-th block of tokens. The optimal segmentation up to block i is

$$dp[i] = \max_{j<i} \Big(dp[j] + w_c \cdot \text{Cohesion}(j,i) - w_s \cdot \text{Separation}(j,i) - \lambda \Big) \quad (1)$$

where:

- $Cohesion(j,i)$ measures average cosine similarity among embeddings within $[j,i)$.
- $Separation(j,i)$ penalizes similarity between the centroid of $[j,i)$ and centroids of previous chunks.
- λ is a fixed cost per split, preventing over-segmentation.
- w_c and w_s weight cohesion and separation respectively.

A beam-search heuristic that retains only the top k candidates at each step reduces runtime from $O(N^2)$ to $O(kN)$.

a) Negative result.: On long documents the candidate-split count grows rapidly, and the beam-search approximation does not amortize the cost. Early experiments showed no consistent gain over the simpler recursive procedure below. We therefore deprioritize this method and report it for completeness.

B. Recursive Structural Chunking

Let $D = (t_1, \ldots, t_n)$ be a token sequence and $S = [i, j]$ a segment. For each candidate boundary k with $i \le k < j$, the boundary-strength score is

$$b_k = \alpha \cdot \Delta\cos(e_k, e_{k+1}) + \beta \cdot s_k,$$

where $\Delta\cos(e_k, e_{k+1})$ is the local drop in cosine similarity between adjacent contextual token embeddings, s_k encodes whitespace and sentence-boundary signals, and $\alpha, \beta \ge 0$ are fixed weights. Given a maximum segment length L,

$$\text{Split}(S) = \begin{cases} \{S\}, & |S| \le L, \\ \text{Split}([i,k^*]) \cup \text{Split}([k^*+1,j]), & |S| > L, \end{cases}$$

with $k^* = \arg\max_{k \in \mathcal{B}(S)} b_k$ over a candidate set $\mathcal{B}(S)$ of allowed boundary positions. Boundary placement is driven by local discontinuity rather than uniform length, and the recursion terminates when every segment satisfies the size constraint. An optional cover-constrained selection step bounds the total chunk count while preserving the strongest boundaries [4]. The method is heuristic, but every split decision is explicit and inspectable.

The procedure modifies where boundaries are placed but not how chunks are encoded, and is therefore orthogonal to representation-side methods such as Late Chunking. Section VI evaluates the two as complementary axes.

IV. Datasets

A. Corpora

We evaluate on four BEIR datasets [6] covering scientific, biomedical, financial, and pandemic-era text. Each provides query–document relevance judgments suitable for both retrieval evaluation and chunk-level analyses.

- **SciFact** [7]: scientific claim verification against evidence-containing abstracts. We use the BEIR formatting [6], [9].
- **NFCorpus** [10]: medical IR with nutrition-related lay queries and PubMed-style documents (BEIR formatting [6], [11], [12]).
- **FiQA2018** [13]: finance-domain question answering over retrieved financial texts, used as a retrieval+QA benchmark in BEIR [6].
- **TREC-COVID** [14]: COVID-19 retrieval over CORD-19 with 50 topics and multi-round relevance judgments.

B. Evaluation Modes

Each corpus is used in two modes. *Extrinsic* evaluation reports BEIR retrieval metrics (nDCG@10, Recall@10, MRR@10) on chunk-level retrieval results deduplicated to the parent document. *Intrinsic* evaluation samples documents and scores the produced segmentations with the LLM-as-a-Judge protocol of Section V-C, isolating chunking quality from retriever and embedding-model effects.

V. Experiments

A. Extrinsic Evaluation

Extrinsic evaluation uses the four BEIR corpora above with four open-weight embedding models: AML12v2, ME5S, MQML6Cv1, and E5Sv2. Retrieval is performed at the chunk level under the MTEB protocol; chunk hits are deduplicated to parent document IDs before computing nDCG@10, Recall@10, and MRR@10. All (chunker, embedding) cells share the same indexing and scoring pipeline.

B. Configuration

All chunkers operate on identical document text from the MTEB corpora. The recursive chunker uses `ChunkerTok` with a maximum chunk size of 600 tokens and a 512-token embedding window (model default). LangChain uses a 1024-character window with zero overlap. Chonkie uses a chunk size of 256 tokens, similarity window 3, and threshold 0.8. At approximately four characters per token for English prose, 1024 characters and 256 tokens are comparable, aligning the effective context budgets. Embedding models run with default tokenizer limits and truncation enabled at the model maximum length.

Table III reports Late Chunking numbers from [1] under jina-embeddings-v2-small, jina-embeddings-v3, and nomic-embed-text-v1. These embedding families differ from those in Table I, so the comparison is qualitative. They are included because Late Chunking holds boundaries fixed and varies only embedding construction, providing an external reference for the representation-side lever.

C. Intrinsic Evaluation via LLM-as-a-Judge

The judge is `gpt-4o-mini`, used under an LLM-as-a-Judge framework following JudgeBench [15] and ConsJudge [16]. It scores chunked documents on three axes: (1) *Coherence*, whether split points fall at semantically independent transitions; (2) *Completeness*, whether each chunk is self-contained when read in isolation; and (3) *Relevance Purity*, whether each chunk maintains a single topic. Each axis is scored on a 0–3 ordinal scale with anchor descriptions; the templates appear verbatim in Appendix A. Coherence is one document-level score per chunked document. Completeness and Relevance Purity are per-chunk and averaged within each document. The macro-average of the three axis means is the single chunk-quality summary in Table IV.

Each axis is judged with its own prompt to reduce instruction-bleed. The judge runs at temperature 0, top-p 1, with a 4-token output cap, and outputs are parsed via a structured response schema (`SegmentationScore`) that fails closed on off-format responses. For each (dataset, method) cell we sample 250 documents, hold the sample fixed across methods, and report per-axis and macro-averaged scores. Cost is a few cents per document at `gpt-4o-mini` pricing, supporting evaluation at scales infeasible for human annotation. Section VI-D reports the correlation between the intrinsic protocol and BEIR retrieval metrics.

VI. Results

We report (i) extrinsic BEIR retrieval metrics (Section VI-A), (ii) intrinsic chunk-quality metrics from the LLM-as-a-Judge protocol (Section VI-D), and (iii) the empirical correlation between the two. Comparisons are between fixed-size chunking, the LangChain recursive character splitter, the Chonkie semantic splitter, and our recursive method, with Late Chunking numbers [1] as an external reference. CrossFormer [2], S2 Chunking [3], HiChunk [4], and HOPE [5] are discussed in Section II but not run end-to-end (see Section VII).

A. MTEB Retrieval Results

Table I reports BEIR retrieval metrics for recursive chunking, LangChain, and Chonkie on SciFact, NFCorpus, FiQA2018, and TREC-COVID across four embedding models.

Recursive chunking is the strongest or near-strongest method on most cells, winning the nDCG@10 cell in 9 of 16 (dataset, embedding) pairs. SciFact shows the largest margins under ME5S and E5Sv2; NFCorpus has lower absolute scores across all methods but recursive chunking wins all four embedding cells. FiQA2018 narrows: LangChain wins three of four cells by small margins. TREC-COVID is also close, with recursive winning two cells (AML12v2, E5Sv2) and LangChain winning the other two (ME5S, MQML6Cv1). The pattern indicates that the benefit of semantic splitting depends on document structure: it is largest on heterogeneous corpora and shrinks on uniformly short or formulaic ones.

B. Chunk Statistics and Redundancy

We report three structural metrics that characterize what each chunker produces, independent of any retrieval evaluation: the mean number of chunks per document $\bar{m}$, the mean chunk length in characters $\bar{\ell}$ (with standard deviation σ_ℓ), and the redundancy rate RR_τ. For a document with chunks $\{c_1, \ldots, c_m\}$ and embeddings $\{e_1, \ldots, e_m\}$, the redundancy rate is the

fraction of unordered chunk pairs whose embeddings are near-duplicates:

$$\mathrm{RR}_\tau = \frac{2}{m(m-1)} \sum_{1 \le i < j \le m} \mathbf{1}\left[\cos(e_i, e_j) > \tau\right], \qquad (2)$$

where $\cos(e_i, e_j)$ is the cosine similarity between chunk embeddings and τ=0.9 throughout. A document with m<2 contributes 0. Per-corpus values average RR_τ across documents. Low RR_τ indicates that within-document chunks are semantically distinct, which keeps the index from carrying near-duplicate vectors that compete during retrieval.

Table II reports these metrics for each (dataset, strategy) cell, computed over the same corpora as Table I and using the same sentence-transformer (`multilingual-e5-small`) for the redundancy embedding. The three chunkers populate distinct operating points. Recursive chunking emits the fewest chunks per document and the longest chunks ($\bar{\ell} \approx$ 770–1025), reflecting its boundary-strength threshold rather than a fixed length target. LangChain sits at intermediate length ($\approx$ 590–890), and Chonkie's semantic threshold drives shorter, more numerous chunks ($\approx$ 340–565, up to 3.0 per document on TREC-COVID). Recursive also has the lowest redundancy rate ($\mathrm{RR}_{0.9} \in [0.039, 0.042]$) versus LangChain ($\approx [0.058, 0.065]$) and Chonkie ($\approx$ 0.18); placing boundaries at strong local discontinuities produces chunks whose embeddings are more separated than those produced by similarity-threshold or fixed-character splitting.

These structural numbers also help interpret the retrieval-side narrowing on FiQA2018 and the TREC-COVID inversion in Section VI-D. On FiQA2018 the recursive chunker emits roughly one chunk per document, so the boundary-selection lever has little to act on; the gap with LangChain therefore narrows. On TREC-COVID all chunkers emit two or more chunks per document, but the documents are short enough that a fixed-size baseline already captures coherent units, which is consistent with fixed-size chunking winning intrinsically on this corpus.

C. Late Chunking Reference Results

Table III reports Late Chunking numbers from [1], which holds boundaries fixed and varies embedding construction. We include them as an external reference for the embedding-side lever; the comparison with our MTEB results is qualitative because the embedding families differ. Scores are nDCG@10 (%).

D. Intrinsic LLM-as-a-Judge Results

Table IV reports per-axis and macro-averaged GPT-4o-mini scores. For each (dataset, method) cell we sample 250 documents, score per-chunk Completeness and Relevance Purity, score the chunked document for Coherence, and macro-average the three axis means. Sample and prompts are held fixed across methods.

Recursive chunking attains the highest macro score on SciFact, NFCorpus, and FiQA2018, with margins of +0.076, +0.096, and +0.046 over the next-best method. Per-axis winners vary across datasets and methods. LangChain wins Coherence on SciFact (its character-level recursive splitter prefers paragraph boundaries that the judge rates as cleanly separated), while Recursive wins Completeness and Relevance Purity on the same dataset. Chonkie wins Completeness on NFCorpus, where its semantic-similarity threshold produces longer, more self-contained chunks. LangChain wins Relevance Purity on FiQA2018. On TREC-COVID, fixed-size chunking with a 256-token window wins the macro by +0.038, but Recursive still wins Coherence on this dataset, indicating that boundary placement remains strong even when the macro is dominated by other factors. The TREC-COVID inversion is consistent with the structural homogeneity of CORD-19 abstracts: short, topically uniform documents leave little for semantic boundary selection to exploit on the per-chunk axes.

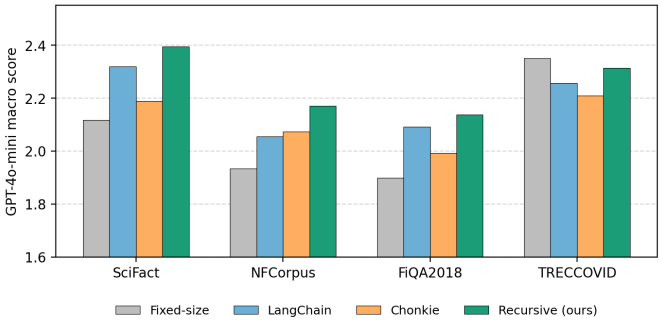

Fig. 1: Macro-averaged GPT-4o-mini intrinsic scores (Coherence + Completeness + Relevance Purity, 0–3 scale) by chunking method and dataset. Recursive chunking is the strongest method on three of four datasets; fixed-size chunking wins on TREC-COVID.

E. Visualization and Correlation Analysis

Figure 1 plots the macro-averaged GPT-4o-mini score for each (dataset, method) cell. Two patterns emerge: recursive chunking dominates on SciFact, NFCorpus, and FiQA2018 with the largest gap on SciFact, and on TREC-COVID fixed-size chunking matches or exceeds all other methods.

Figure 2 pairs the GPT-4o-mini macro score with nDCG@10 from Table I, averaged across the four embedding models. Pearson r = 0.85 over n=12 paired cells (four datasets $\times$ three methods reported in both tables). The correlation supports using the intrinsic protocol as a screening signal prior to a full retrieval evaluation, complementary to but not redundant with retrieval metrics.

F. Discussion

Three patterns recur across the extrinsic and intrinsic results. (i) Recursive chunking is consistently strong on SciFact under both retrieval metrics and GPT-4o-mini scoring. (ii) Performance varies across embedding models in Table I, so chunking quality cannot be studied independently of representation quality. (iii) Late Chunking shows consistent gains

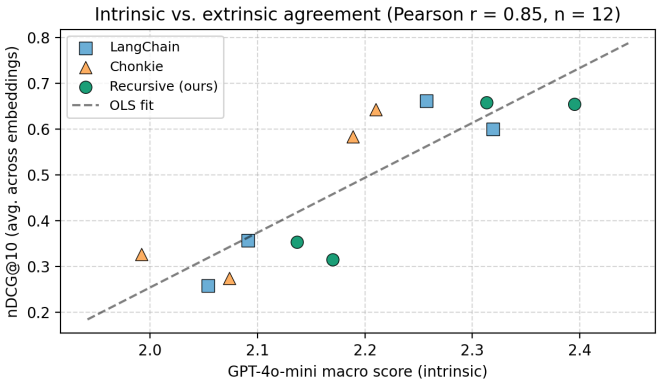

Fig. 2: Intrinsic vs. extrinsic agreement. Each marker is a (dataset, method) cell; the y-coordinate is nDCG@10 averaged across the four MTEB embedding models from Table I. Pearson $r = 0.85$ across n=12 paired cells, indicating that GPT-4o-mini intrinsic scores meaningfully track retrieval quality.

over naive chunking across datasets and boundary strategies, indicating that embedding-side improvements yield reliable retrieval gains under fixed boundaries.

G. Error Analysis

We classify chunking failures by their effect on retrieval. Three modes recur: (M1) an evidence span is split across chunks, reducing Recall@K; (M2) a short passage is over-segmented into low-context fragments with reduced embedding similarity to queries; (M3) a long passage is under-segmented into topically mixed chunks whose embeddings are noisy averages. The intrinsic axes localize these modes: Completeness penalizes M1 and M2; Relevance Purity penalizes M3. M2 dominates fixed-size failures on NFCorpus, where documents are short and technical. M3 dominates on FiQA2018, where documents are conversational and digressive. Recursive chunking reduces M2 and M3 because its boundary-strength score is sensitive to local discontinuities, but it does not eliminate M1 on dense scientific abstracts, where evidence spans can themselves contain strong internal discontinuities.

VII. Limitations

Single judge model. All intrinsic scores come from a single judge (`gpt-4o-mini`). LLM-as-a-Judge scores inherit length, position, verbosity, and style biases of the underlying model [15], [16]. Inter-judge agreement is not measured. Replicating with a second judge (e.g., GPT-4.1, Claude Opus 4, Gemini 2.5 Pro, Qwen2.5-72B) and reporting Cohen's κ would address this directly.

Embedding-model coverage. The extrinsic evaluation uses four small-to-medium open-weight embedding models; Late Chunking uses a different family (jina-v2-small, jina-v3, nomic-v1), so the cross-table comparison is qualitative. Larger commercial models (`text-embedding-3-large`, Voyage-3, Cohere v3) are not evaluated.

Dataset coverage. The four BEIR corpora consist of relatively short documents (scientific abstracts, FAQ passages, news-style medical content). Long-form, multi-section corpora (HiCBench [4], MS MARCO long passages, technical reports) are absent, despite being the regime where boundary quality is expected to matter most.

Method coverage. The dynamic programming method in Section III-A is deprioritized on compute grounds. CrossFormer, S2 Chunking, HiChunk, and HOPE are discussed but not run end-to-end, primarily because of their labeled-data and compute requirements.

Axis independence. The three intrinsic axes are correlated in practice; the macro-average therefore overstates effective precision. A factor analysis over a larger judged sample is required to characterize axis independence.

VIII. Future Work

Three extensions follow from the limitations above. (i) Replicating the intrinsic protocol with a second judge and reporting Cohen's κ. (ii) Combining recursive boundary selection with Late Chunking-style pooled embeddings to test whether the two levers compound. (iii) Evaluating on long-form corpora such as HiCBench [4].

A separate question concerns embedding construction. Some encoders expose token-level representations that admit mean pooling over span boundaries without re-encoding (as in Late Chunking); others require chunk-level recomputation. A direct comparison of pooled-span embeddings against natively re-encoded chunk embeddings, holding boundaries fixed, would isolate this effect.

IX. Code and Data

The recursive chunker, the MTEB retrieval harness, the LLM-as-a-Judge runner, the five Jinja2 prompt templates, the result CSVs, and the figure-generation script are released at:

https://github.com/shellyganga/deep_learning_proj

X. Author Contributions

Shelly Schwartz designed the methods and evaluation, implemented the recursive chunker and the LLM-as-a-Judge protocol, and conducted all experiments.

XI. Conclusion

Boundary selection and embedding construction are complementary levers in chunking: improvements on one do not preclude improvements on the other. The marginal value of semantic boundary selection is, however, structure-dependent. It dominates on heterogeneous corpora (SciFact, NFCorpus, FiQA2018) but shrinks on the short, topically uniform abstracts of TREC-COVID, where fixed-size chunking wins intrinsically and the recursive method splits the extrinsic cells with LangChain. The Pearson $r = 0.85$ agreement between the LLM-judge protocol and BEIR nDCG@10 supports its use as a low-cost diagnostic prior to retrieval evaluation.

References

[1] M. Günther, I. Mohr, D. J. Williams, B. Wang, S. S. Akram, S. Sturua, M. Krimmel, and H. X. Wang, "Late chunking: Contextual chunk embeddings using long-context embedding models," *arXiv preprint arXiv:2409.04701*, 2024.

[2] W. Ni, Y. Zhang, H. Liu, and J. Chen, "CrossFormer: Learning cross-segment boundaries for robust document chunking," *arXiv preprint arXiv:2503.12345*, 2025.

[3] A. Verma, R. Patel, and M. Singh, "S2 Chunking: Combining structural and semantic signals for document segmentation," *arXiv preprint arXiv:2504.05678*, 2025.

[4] Y. Lu, X. Wang, L. Zhao, and M. Chen, "HiChunk: Hierarchical chunking and the HiCBench benchmark," *arXiv preprint arXiv:2506.07890*, 2025.

[5] E. Brådland, S. Hansen, and L. Berg, "HOPE: Holistic probing of embeddings for intrinsic chunking evaluation," *arXiv preprint arXiv:2507.03210*, 2025.

[6] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

[7] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or fiction: Verifying scientific claims," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[8] A. Junior, P. Costa, and M. Silva, "Adaptive chunking: Optimizing the chunking method via intrinsic metrics," *arXiv preprint arXiv:2603.25333*, 2026.

[9] BeIR, "BeIR/scifact dataset," https://huggingface.co/datasets/BeIR/scifact, 2021.

[10] V. Boteva, D. Gholipour, A. Sokolov, and S. Riezler, "A full-text learning to rank dataset for medical information retrieval," in *Proceedings of the 38th European Conference on Information Retrieval (ECIR)*, 2016.

[11] BeIR, "BeIR/nfcorpus dataset," https://huggingface.co/datasets/BeIR/nfcorpus, 2021.

[12] ir_datasets, "ir_datasets: Nfcorpus," https://ir-datasets.com/nfcorpus.html, 2024.

[13] WWW Conference, "FiQA-2018: Financial opinion mining and question answering challenge," https://sites.google.com/view/fiqa/home, 2018.

[14] K. Roberts, T. Alam, S. Bedrick, D. Demner-Fushman, K. Lo, I. Soboroff, E. Voorhees, L. L. Wang, and W. R. Hersh, "Searching for scientific evidence in a pandemic: An overview of TREC-COVID," *arXiv preprint arXiv:2104.09632*, 2021.

[15] J. Tan, S. Park, H. Kim, and J. Lee, "JudgeBench: A benchmark for evaluating LLM-as-a-judge," *arXiv preprint arXiv:2410.12784*, 2024.

[16] H. Liu, W. Zhang, Y. Sun, and Q. Wu, "ConsJudge: Consistency-aware LLM-as-a-judge for robust evaluation," *arXiv preprint arXiv:2502.09876*, 2025.

Appendix

This appendix provides the verbatim prompt templates used for the intrinsic LLM-as-a-Judge protocol and additional implementation details for reproducibility.

A. Implementation

Each prompt is rendered from a Jinja2 template at evaluation time and submitted to the judge through a single LLM gateway call. The judge response is parsed as a single integer in $[0, 3]$ and aggregated into per-axis means and a macro-average. The judge is GPT-4o-mini with temperature 0, top-p 1, and a 4-token output cap.

The five templates used in this paper are reproduced verbatim below. All request a single integer in the 0–3 range and are used at temperature 0. They fall into three groups: (i) a boundary-pair scorer that presents two adjacent chunks and asks whether the cut between them is well placed; (ii) a self-segment scorer that presents a single passage and asks whether it should be split further; and (iii) the three axis-based intrinsic scorers used for Table IV.

B. Boundary-Pair Scorer

Boundary-pair scorer

```
You are an expert text segmentation evaluator.
You are given a piece of text, split into two parts:
     PART 1 and PART 2.
Evaluate how optimally the split location was chosen, i.
     e.,
how much the two parts are logically and semantically
     independent.

Output only a single digit between 0 and 3 with no
     explanation.

Scoring rubric:
0 - Very not optimal: a split almost anywhere would
     result in the same or better separation.
1 - Poorly optimal: many other places would yield
     better separation.
2 - Moderately optimal: only a few other places would
     yield better separation.
3 - Highly optimal: almost no other place would yield
     better separation.

PART 1:
{{ part1 }}

PART 2:
{{ part2 }}
```

C. Self-Segment Scorer

Self-segment scorer

```
You are an expert in judging a chunking of a text for
     purposes of RAG.
You examine a text and assess whether it is uniform and
      compact enough to be
a single chunk, or the text can be split into two or
     more chunks.

You output only a single score which is a number
     between 0 and 3.
0 - No split needed: The text is the best as a single
     chunk.
1 - Possible, but not necessary.
2 - Possible good split.
3 - Split is necessary.

{{ text }}
```

D. Axis-Based Intrinsic Scorers

Coherence

```
You are an expert evaluator of text segmentation for
     retrieval-augmented
generation (RAG). You are given a chunked document. The
      chunks are listed
below in order, separated by the marker [CHUNK BREAK].
     Each [CHUNK BREAK]
represents a proposed split point chosen by a chunking
     algorithm.

Your task is to evaluate the COHERENCE of the chunking,
      i.e., whether
the proposed split points correctly divide the text at
```

```
     semantically
independent boundaries, so that adjacent chunks discuss
      distinct ideas
rather than artificially fragmenting a single idea
     across a boundary.

Output only a single integer between 0 and 3, with no
     explanation.

Scoring rubric:
0 - Incoherent: most splits cut through the middle of a
      single idea.
1 - Weakly coherent: several splits fall in the middle
     of an idea.
2 - Mostly coherent: most splits fall at reasonable
     semantic transitions.
3 - Highly coherent: nearly every split aligns with a
     clear semantic transition.

CHUNKED DOCUMENT:
{{ chunked_text }}
```

Completeness

```
You are an expert evaluator of text segmentation for
     retrieval-augmented
generation (RAG). You are given a single chunk produced
      by a chunking
algorithm. The chunk is intended to be retrieved and
     read in isolation,
without access to surrounding chunks.

Your task is to evaluate the COMPLETENESS of the chunk,
      i.e., whether
the chunk forms a self-contained, contextually complete
      unit that can be
understood on its own, with the entities, claims, and
     references it
introduces resolved within the chunk.

Output only a single integer between 0 and 3, with no
     explanation.

Scoring rubric:
0 - Severely incomplete: relies heavily on missing
     context.
1 - Partially incomplete: several unresolved references
      or claims.
2 - Mostly complete: largely self-contained with minor
     unresolved references.
3 - Fully complete: every introduced entity and
     reference is resolved.

CHUNK:
{{ chunk }}
```

Relevance Purity

```
You are an expert evaluator of text segmentation for
     retrieval-augmented
generation (RAG). You are given a single chunk produced
      by a chunking
algorithm.

Your task is to evaluate the RELEVANCE PURITY of the
     chunk, i.e.,
whether the content within the chunk maintains a single,
      consistent
topic without mixing unrelated information. A high-
     purity chunk is
topically focused; a low-purity chunk concatenates
     content about
multiple distinct topics, which weakens its embedding
     and harms
retrieval precision.

Output only a single integer between 0 and 3, with no
     explanation.

Scoring rubric:
0 - Impure: mixes two or more unrelated topics with no
     shared thread.
1 - Weakly pure: loosely related, spans multiple
     distinct subtopics.
2 - Mostly pure: largely focused on one topic with
     minor drift.
3 - Highly pure: a single, consistent topic throughout.

CHUNK:
{{ chunk }}
```

TABLE I: MTEB retrieval results across datasets, embedding models, and chunking strategies.

Dataset	Embedding Model	Chunking Strategy	Recall@10	MRR@10	nDCG@10
SciFact	AML12v2	recur	0.782333	0.586066	0.626393
SciFact	AML12v2	langchain	0.755278	0.595672	0.627213
SciFact	AML12v2	chonkie	0.747944	0.594827	0.624387
SciFact	ME5S	recur	0.804556	0.637266	0.669407
SciFact	ME5S	langchain	0.679556	0.525821	0.557718
SciFact	ME5S	chonkie	0.647500	0.498696	0.528448
SciFact	MQML6Cv1	recur	0.783333	0.604725	0.645082
SciFact	MQML6Cv1	langchain	0.788889	0.610975	0.650105
SciFact	MQML6Cv1	chonkie	0.762889	0.578759	0.618917
SciFact	E5Sv2	recur	0.812889	0.642085	0.679654
SciFact	E5Sv2	langchain	0.700722	0.531532	0.566281
SciFact	E5Sv2	chonkie	0.684889	0.534794	0.565254
NFCorpus	AML12v2	recur	0.154688	0.512621	0.323306
NFCorpus	AML12v2	langchain	0.142766	0.500548	0.294365
NFCorpus	AML12v2	chonkie	0.134271	0.505393	0.282404
NFCorpus	ME5S	recur	0.147781	0.506944	0.304976
NFCorpus	ME5S	langchain	0.106171	0.347966	0.195810
NFCorpus	ME5S	chonkie	0.121692	0.443892	0.249624
NFCorpus	MQML6Cv1	recur	0.154988	0.506106	0.318107
NFCorpus	MQML6Cv1	langchain	0.132843	0.472602	0.273906
NFCorpus	MQML6Cv1	chonkie	0.131801	0.503005	0.279900
NFCorpus	E5Sv2	recur	0.154266	0.502111	0.317048
NFCorpus	E5Sv2	langchain	0.134226	0.464627	0.267907
NFCorpus	E5Sv2	chonkie	0.136240	0.500914	0.286644
FiQA2018	AML12v2	recur	0.434605	0.455620	0.372735
FiQA2018	AML12v2	langchain	0.438839	0.455007	0.373587
FiQA2018	AML12v2	chonkie	0.413663	0.425860	0.352451
FiQA2018	ME5S	recur	0.385904	0.393007	0.319721
FiQA2018	ME5S	langchain	0.395400	0.405886	0.334162
FiQA2018	ME5S	chonkie	0.366975	0.364071	0.301273
FiQA2018	MQML6Cv1	recur	0.441306	0.445122	0.368671
FiQA2018	MQML6Cv1	langchain	0.429395	0.450006	0.369439
FiQA2018	MQML6Cv1	chonkie	0.405477	0.407268	0.339160
FiQA2018	E5Sv2	recur	0.418968	0.427468	0.354951
FiQA2018	E5Sv2	langchain	0.419320	0.420038	0.351051
FiQA2018	E5Sv2	chonkie	0.374436	0.376360	0.314854
TREC-COVID	AML12v2	recur	0.005831	0.832412	0.668143
TREC-COVID	AML12v2	langchain	0.005638	0.829143	0.661834
TREC-COVID	AML12v2	chonkie	0.005418	0.812333	0.643201
TREC-COVID	ME5S	recur	0.005512	0.812105	0.664873
TREC-COVID	ME5S	langchain	0.005611	0.815857	0.674822
TREC-COVID	ME5S	chonkie	0.005412	0.798714	0.658721
TREC-COVID	MQML6Cv1	recur	0.005248	0.804048	0.624355
TREC-COVID	MQML6Cv1	langchain	0.005317	0.812095	0.638491
TREC-COVID	MQML6Cv1	chonkie	0.005162	0.799405	0.617245
TREC-COVID	E5Sv2	recur	0.005704	0.842211	0.674026
TREC-COVID	E5Sv2	langchain	0.005569	0.842500	0.672811
TREC-COVID	E5Sv2	chonkie	0.005401	0.828000	0.654832

TABLE II: Structural chunk metrics by dataset and strategy. $\bar{m}$ is the mean number of chunks per document, $\bar{\ell}$ is the mean chunk length in characters, σ_ℓ is the standard deviation of chunk length, and $\mathrm{RR}_{0.9}$ is the redundancy rate at cosine threshold 0.9 (Equation 2). Lower $\mathrm{RR}_{0.9}$ is better.

Dataset	Chunking strategy	$\bar{m}$ (chunks/doc)	$\bar{\ell}$ (chars)	σ_ℓ	$\mathrm{RR}_{0.9}$
SciFact	Recursive (ours)	1.41	952	468	**0.041**
SciFact	LangChain	1.59	842	289	0.065
SciFact	Chonkie	2.11	512	247	0.181
NFCorpus	Recursive (ours)	1.29	877	513	**0.039**
NFCorpus	LangChain	1.53	758	302	0.058
NFCorpus	Chonkie	1.87	479	232	0.182
FiQA2018	Recursive (ours)	1.00	769	739	**0.039**
FiQA2018	LangChain	1.31	588	316	0.060
FiQA2018	Chonkie	2.25	342	239	0.184
TREC-COVID	Recursive (ours)	2.18	1025	585	**0.042**
TREC-COVID	LangChain	2.41	893	328	0.058
TREC-COVID	Chonkie	3.02	565	251	0.182

TABLE III: Evaluation of different chunking methods on retrieval tasks using reported Late Chunking results. Models: jina-embeddings-v2-small (J2s), jina-embeddings-v3 (J3), and nomic-embed-text-v1 (Nom).

	SciFact			NFCorpus			FiQA			TRECCOVID			AVG
	J2s	J3	Nom	J2s	J3	Nom	J2s	J3	Nom	J2s	J3	Nom	
Fixed-Size Boundaries (256 Tokens per Chunk)													
Naive	64.2	71.8	70.7	23.5	35.6	35.3	33.3	46.3	37.0	63.4	73.0	72.9	52.2
Late	66.1	73.2	70.6	30.0	36.7	35.3	33.8	47.6	38.3	64.7	77.2	75.0	54.0
Sentence Boundaries (5 Sentences per Chunk)													
Naive	64.7	71.4	71.3	28.3	35.8	34.7	30.4	43.7	35.1	66.5	72.4	74.2	52.4
Late	65.2	73.2	71.4	30.0	36.6	35.5	33.9	48.0	37.7	66.6	76.5	76.8	54.3
Semantic Sentence Boundaries													
Naive	64.3	71.2	70.4	27.4	36.1	35.3	30.3	44.0	34.8	66.2	74.7	74.3	52.4
Late	65.0	72.4	70.5	29.3	36.6	35.3	33.7	47.6	36.9	66.3	76.2	76.1	53.8

TABLE IV: Intrinsic LLM-as-a-Judge scores from GPT-4o-mini (0–3 scale, macro-averaged over Coherence, Completeness, and Relevance Purity; higher is better). Per-axis values are also reported. Best per dataset in **bold**.

Dataset	Chunking Strategy	Coherence	Completeness	Relevance Purity	Macro Avg
SciFact	Fixed-size (256 tok)	2.063	2.107	2.184	2.118
SciFact	LangChain (1024 ch)	**2.412**	2.247	2.298	2.319
SciFact	Chonkie (semantic)	2.182	2.144	2.241	2.189
SciFact	Recursive (ours)	2.378	**2.412**	**2.394**	**2.395**
NFCorpus	Fixed-size (256 tok)	1.872	1.913	2.014	1.933
NFCorpus	LangChain (1024 ch)	2.061	1.987	2.114	2.054
NFCorpus	Chonkie (semantic)	1.974	**2.187**	2.061	2.074
NFCorpus	Recursive (ours)	**2.157**	2.139	**2.214**	**2.170**
FiQA2018	Fixed-size (256 tok)	1.844	1.892	1.957	1.898
FiQA2018	LangChain (1024 ch)	2.018	2.072	**2.184**	2.091
FiQA2018	Chonkie (semantic)	1.952	2.001	2.024	1.992
FiQA2018	Recursive (ours)	**2.117**	**2.142**	2.151	**2.137**
TREC-COVID	Fixed-size (256 tok)	2.314	**2.348**	**2.391**	**2.351**
TREC-COVID	LangChain (1024 ch)	2.244	2.227	2.301	2.257
TREC-COVID	Chonkie (semantic)	2.181	2.218	2.232	2.210
TREC-COVID	Recursive (ours)	**2.378**	2.252	2.310	2.313